

# LAB-4: LAVA ESCAPE

LLM Chat / Pair-Programming History

**Purpose:** Documentation of the iterative LLM-assisted debugging and implementation process used while completing Tasks 1–4 of the Lava Escape lab.

**Important:** This document is a structured submission record based on the actual conversation workflow. It is not presented as a verbatim export of every ChatGPT message.

## Submission Requirements Referenced

The Lab-4 instructions require the submission to include the complete Chat/LLM history, with the chat history exported as a document/PDF. The lab also requires before/after gameplay videos and updated code.

## Workflow Used

The LLM was used as a debugging/pair-programming partner. The workflow followed the requested sequence: inspect the existing code, implement one task at a time, run/test the game, troubleshoot if necessary, and then continue to the next task.

## Task 1 — Platform Collision Fix

User goal:

Complete Task 1 first and verify whether the existing implementation satisfies the requirement.

Initial implementation inspected:

The `Player.update()` method stored the player's previous bottom position before vertical movement and used a one-way platform collision condition.

Important logic:

```
previous_bottom = self.rect.bottom
```

```
self.rect.y += int(self.vel_y)
```

```
...
```

```
if self.rect.colliderect(p) and self.vel_y > 0 and previous_bottom <= p.top + 2:
```

```
self.rect.bottom = p.top
```

```
self.vel_y = 0
```

```
self.on_ground = True
```

LLM guidance:

The collision check was reviewed to ensure the player only lands when falling and when the player's previous feet position was at or above the platform top.

Testing:

The game was run after the implementation. Task 1 was treated as completed before proceeding to Task 2.

## Task 2 — Crumbling Platforms

User goal:

Implement Task 2 only. Selected platforms must become unstable; when the player lands on one, a one-second countdown/shaking animation occurs and the platform is removed from `self.platforms`. Task 3 and Task 4 were explicitly not to be modified during this stage.

**Implementation approach:**

**A Platform class was introduced so each platform could store state.**

**Task 2 state:**

```
self.crumbling = crumbling
```

```
self.crumble_started_at = None
```

When the player lands:

```
if p.crumbling and p.crumble_started_at is None:
```

```
p.crumble_started_at = pygame.time.get_ticks()
```

The game engine then checks elapsed time and removes the platform after 1000 ms:

```
if elapsed >= CRUMBLE_DURATION:
```

```
continue
```

The drawing code also offsets the platform horizontally during the countdown to create a shaking effect.

Testing:

The crumbling behavior was checked as: land → shake/countdown → disappear. Task 2 was completed before moving to Task 3.

## Task 3 — Spring Bounce Platforms

User goal:

Implement Task 3 only after Task 2 was completed. Selected platforms should be spring platforms, visually distinct

and yellow, and should launch the player upward with a stronger bounce.

#### **Implementation approach:**

**Platform state was extended with:**

**self.spring = spring**

**self.spring\_started\_at = None**

Spring platforms were generated separately from crumbling platforms. The implementation prevented a platform from being both crumbling and spring.

#### **Spring launch:**

**SPRING\_JUMP\_VELOCITY = -22**

When the player landed on a spring:

if p.spring:

self.vel\_y = SPRING\_JUMP\_VELOCITY

self.on\_ground = False

A short squash/recoil visual effect was also added to the player.

#### **Visual design:**

**Spring platforms were assigned a bright yellow color and spring-coil lines were drawn on them.**

Testing:

The game was run and the spring platform behavior was checked: land on yellow spring → strong upward launch.

Task 3 was then committed/pushed to the user's repository.

## **Task 4 — Lava Danger Meter and Lava Burst**

User goal:

Implement Task 4 only, while keeping Tasks 1–3 intact.

Required behavior:

1. Display a danger meter related to lava rise/speed.
2. Trigger a Lava Burst every 15 seconds.
3. During the burst, lava rises faster for 3 seconds.
4. Return to normal lava behavior after the burst.

#### **Implementation constants:**

**LAVA\_BURST\_INTERVAL = 15000**

**LAVA\_BURST\_DURATION = 3000**

**LAVA\_BURST\_MULTIPLIER = 3.0**

#### **Timing:**

The game records `game_start_time` and uses `pygame.time.get_ticks()` to determine when 15 seconds have elapsed.

#### **During a burst:**

**self.lava\_rise = self.normal\_lava\_rise \* LAVA\_BURST\_MULTIPLIER**

#### **After 3 seconds:**

**self.lava\_burst\_active = False**

**self.lava\_burst\_started\_at = None**

**self.lava\_rise = self.normal\_lava\_rise**

**Danger meter:**

**A HUD meter was drawn at the top-right and its fill was calculated from the current lava rise speed.**

Testing:

The game was run and Task 4 was checked by observing the danger meter and waiting for the Lava Burst event. After Task 4 was completed, all four tasks were considered complete.

## Final Verification / Submission Record

| Requirement                        | Status                               |
|------------------------------------|--------------------------------------|
| Task 1 — Platform collision        | Completed                            |
| Task 2 — Crumbling platforms       | Completed                            |
| Task 3 — Spring bounce platforms   | Completed                            |
| Task 4 — Lava danger meter + burst | Completed                            |
| Before gameplay video              | Recorded                             |
| After gameplay video               | Recorded                             |
| Updated code                       | Pushed to personal GitHub repository |
| LLM / Chat history                 | This PDF                             |

**Final workflow:** Task 1 → Task 2 → Task 3 → Task 4 → testing → individual code commits/pushes → before/after videos → LLM history documentation.

**Note:** This PDF summarizes the actual iterative chat workflow visible in the conversation. If the evaluator explicitly requires a literal platform-exported transcript, retain the original ChatGPT conversation/export as well.